GUIDE DE TEST DU PROJET

Répondeur Téléphonique Intelligent — Cabinet Dentaire

Pipeline : STT (Vosk) → LLM (Mistral 7B) → TTS (edge-tts)

Projet	saif-pfe-repondeur-intelligent
GitHub	github.com/saadaoui123/saif-pfe-repondeur-intelligent
Étudiant	Saif — PFE ISET / DKSoft
Date	Mars 2026
Environnement	Ubuntu Server 24 | Python 3.12 | stt_env

1. Prérequis Système

1.1 Configuration minimale

Composant	Exigence
OS	Ubuntu 22.04 / 24.04
Python	3.8 ou supérieur
RAM	8 Go minimum — 16 Go recommandé
Disque	10 Go minimum

1.2 Vérification Python

```
python3 --version
sudo apt update
sudo apt install -y curl git unzip net-tools
```

2. Installation du Projet

2.1 Cloner le dépôt GitHub

```
git clone https://github.com/saadaoui123/saif-pfe-repondeur-intelligent.git
cd saif-pfe-repondeur-intelligent
# OU si vous êtes déjà dans saif_pfe :
cd ~/saif_pfe
```

2.2 Créer l'environnement virtuel

```
python3 -m venv stt_env
source stt_env/bin/activate
which python
# Doit afficher : /home/saif/saif_pfe/stt_env/bin/python
```

2.3 Installer les dépendances Python

```
pip install --upgrade pip
pip install vosk websockets requests numpy pytest
```

2.4 Installer Ollama et Mistral 7B

```
# Installer Ollama
curl -fsSL https://ollama.com/install.sh | sh
ollama --version

# Télécharger le modèle Mistral (~2.1 Go)
ollama pull mistral
ollama list
# Doit afficher : mistral:latest
```

2.5 Configurer Zoiper5

Télécharger et installer Zoiper5, puis configurer le compte SIP :

- Domain : 1001@ IP address de votre machine madame .
- Port : 5060
- Utilisateur : 1001
- Mot de passe : 1234
- Numéro à appeler : 700

2.6 Structure des modèles Vosk

```
ls -la models/
# Doit afficher :
# vosk-model-fr-0.22.zip
# vosk-model-small-fr-0.22.zip

# Décompresser le modèle de test :
unzip models/vosk-model-small-fr-0.22.zip -d models/
```

3. Configuration

3.1 Variables d'environnement

```
echo 'export OLLAMA_HOST="http://localhost:11434"' >> ~/.bashrc
echo 'export OLLAMA_MODEL="mistral"' >> ~/.bashrc
echo 'export VOSK_MODEL_PATH="models/vosk-model-small-fr-0.22"' >> ~/.bashrc
echo 'export SIP_PORT="5060"' >> ~/.bashrc
source ~/.bashrc
```

3.2 Fichier config.py

```
# config.py
OLLAMA_CONFIG = {
    "host": "http://localhost:11434",
    "model": "mistral",
    "temperature": 0.3
}
```

```
VOSK_CONFIG = {
    "model_path": "models/vosk-model-small-fr-0.22",
    "sample_rate": 8000
}
SIP_CONFIG = {
    "port": 5060,
    "domain": "localhost"
}
```

4. Démarrage du Système

Ouvrir 3 terminaux séparés et exécuter dans l'ordre :

Étape 1 — Terminal 1 : Démarrer Ollama

```
ollama serve
# Le serveur écoute sur http://localhost:11434
```

Étape 2 — Terminal 1 (vérification) : Tester Ollama

```
curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "Bonjour, comment puis-je vous aider ?"
}'
```

Étape 3 — Terminal 2 : Démarrer le serveur AudioSocket (Vosk)

```
cd ~/saif_pfe
source ~/stt_env/bin/activate
python stt/audiosocket_vosk_server.py
# Le serveur écoute sur le port 6000
```

Étape 4 — Terminal 3 : Lancer l'application principale

```
cd ~/saif_pfe
source ~/stt_env/bin/activate
python main.py
```

Étape 5 — Tester avec Zoiper5

1. Ouvrir Zoiper5 et se connecter avec le compte SIP 1001
2. Appeler le numéro 700
3. Après le bip, parler clairement en français
4. Écouter la réponse générée par DENTA (Mistral 7B + TTS)

5. Tests du Projet

5.1 Test de reconnaissance vocale (Vosk)

```
cd ~/saif_pfe
```

```
source ~/stt_env/bin/activate
python tests/vosk_client_test.py
```

5.2 Test Ollama en ligne de commande

```
ollama run mistral "Quels sont vos horaires d'ouverture ?"
```

5.3 Test de l'API REST Ollama

```
curl -X POST http://localhost:11434/api/generate \
  -H "Content-Type: application/json" \
  -d '{
    "model": "mistral",
    "prompt": "Bonjour, je voudrais prendre rendez-vous",
    "stream": false
  }'
```

5.4 Test des scénarios de contexte (Prompt Engineering)

Ce test valide les 158 scénarios chargés depuis scenarios.csv — détection d'intent + réponse DENTA :

```
cd ~/saif_pfe
source ~/stt_env/bin/activate
python tests/test_context.py
```

Résultats attendus pour chaque scénario :

Phrase patient	Intent attendu	Hint	Réponse
"Bonjour je veux un rendez-vous j'ai mal aux dents"	prise_rdv	✓	✓
"C'est urgent j'ai très mal"	urgence	✓	✓
"Vous êtes ouverts le samedi ?"	horaires	✓	✓
"J'ai un gonflement de la joue et de la fièvre"	triage_gonflement_visage	✓	✓
"Est-ce que vous acceptez les enfants ?"	info_cabinet	✓	✓

5.5 Test complet du pipeline

```
python -c "
from stt.audiosocket_vosk_server import VoiceRecognizer
from llm.ollama_client import ask_mistral
from llm.context_loader import load_scenarios, detect_intent

scenarios = load_scenarios()
print(f'✓ Scénarios chargés : {len(scenarios)}')

reponse = ask_mistral([{'role': 'user', 'content': 'Bonjour'}])
print(f'✓ LLM répond : {reponse[:50]}...')
"
```

6. Architecture du Projet

6.1 Structure des fichiers

```
/home/saif/saif_pfe/
├── stt_env/                # Environnement virtuel Python
├── models/                 # Modèles Vosk STT
│   ├── vosk-model-fr-0.22.zip
│   └── vosk-model-small-fr-0.22.zip
├── stt/                    # Speech-to-Text
│   └── audiosocket_vosk_server.py
├── llm/                    # Interface LLM
│   ├── __init__.py
│   ├── cabinet_config.py
│   ├── context_loader.py
│   ├── ollama_client.py
│   └── prompt_builder.py
├── tts/                    # Text-to-Speech
├── tests/                  # Tests
│   ├── vosk_client_test.py
│   ├── debug_server.py
│   └── test_context.py
├── scenarios.csv           # 158 scénarios patient
├── config.py
├── main.py
└── README.md
```

6.2 Flux de traitement

Étape	Composant	Description	Technologie
1	Appel entrant	Patient appelle via réseau SIP	Zoiper5 + Asterisk
2	Capture audio	Flux PCM 8000 Hz via AudioSocket TCP	AudioSocket port 6000
3	STT	Transcription voix → texte en temps réel	Vosk fr-0.22
4	Intent + Contexte	Détection intention patient + hint CSV	context_loader.py
5	LLM	Génération réponse médicale adaptée	Mistral 7B / Ollama
6	TTS	Synthèse vocale française	edge-tts (Denise FR)
7	Réponse	Audio envoyé au patient via Asterisk	AudioSocket → Zoiper5

7. Dépannage

Problème : Ollama not found

```
which ollama
# Si non trouvé :
curl -fsSL https://ollama.com/install.sh | sh
```

Problème : Cannot connect to Ollama

```
ps aux | grep ollama
# Si pas de processus :
ollama serve
netstat -tulpn | grep 11434
```

Problème : Modèle Mistral non trouvé

```
ollama list
# Si mistral absent :
ollama pull mistral
```

Problème : Vosk ne trouve pas le modèle

```
ls -la models/
export VOSK_MODEL_PATH="models/vosk-model-small-fr-0.22"
```

Problème : AudioSocket Connection refused

```
# Vérifier que le serveur tourne :
ps aux | grep audiosocket
# Relancer :
source ~/stt_env/bin/activate
python stt/audiosocket_vosk_server.py
```

Problème : Zoiper ne se connecte pas

```
sudo netstat -tulpn | grep 5060
tail -f ~/.Zoiper5/zoiper.log
```

Problème : Audio qui ne fonctionne pas

```
arecord -l
aplay -l
arecord -d 5 test.wav && aplay test.wav
```

8. Checklist de Validation

Test	Statut	Notes
Environnement virtuel stt_env actif	✓ OK	
Ollama démarré (port 11434)	✓ OK	
Modèle mistral:latest installé	✓ OK	
Serveur AudioSocket port 6000 actif	✓ OK	
158 scénarios chargés depuis scenarios.csv	✓ OK	
Détection intent prise_rdv correcte	✓ OK	
Réponse DENTA générée par Mistral	✓ OK	
Zoiper5 connecté SIP 1001	✓ OK	

Appel complet patient → DENTA fonctionnel	✓ OK	
Intégration RAG (prochaine phase)	PLANIFIÉ	